PRÁCTICA 4

"Manipulación de contraste en imágenes digitales: operaciones locales y globales basadas en procesamiento de histogramas"

OBJETIVO: Ilustrar e implementar técnicas de procesamiento imágenes que tienen como objetivo la manipulación del contraste en imágenes digitales.

La práctica se ha dividido en tres partes:

1. **Primera parte:** se implementarán transformaciones locales para manipular el contraste de determinadas zonas del histograma.

2. **Segunda parte:** se implementará la técnica global de ecualización uniforme o igualación del histograma para realzar una imagen. Se ilustrará la aplicación de la técnica cuando se implementa de forma global en toda la imagen, de forma zonal y de forma local a nivel de píxel y por ventanas.

3. **Tercera parte:** se aplicará la técnica de especificación de histogramas con el objetivo de uniformizar el color de distintas imágenes de fondo de ojo.

Material teórico: documentación teórica facilitada del tema 3 "Procesamiento de imágenes: realzado y suavizado" (apartados 3.1. y 3.2).

Primera Parte. Manipulación de contraste mediante transformaciones locales.

1. Utilizando como entrada la imagen *img.png* facilitada, obtener distintas imágenes de salida resultados de la aplicación de las siguientes transformaciones locales de manipulación de contraste:
   - Amplitud de Contraste aplicada como una función lineal definida a trozos que aumente el contraste en la parte baja del histograma. La transformación se compone de dos funciones lineales definidas por los puntos *(p,q) = { (0,0), (50,200), (255,255) }*, donde *p* y *q* denotan el nivel de gris de entrada y salida, respectivamente.
   - Funciones Cuadrada y Cúbica.
   - Funciones Raíz Cuadrada y Raíz Cúbica.
   - Funciones Sigmoide (con $\alpha = 0.85$).
2. Visualice en distintas ventanas gráficas, las imágenes de entrada-salida y sus histogramas.

Observación:

- Se deben describir las técnicas según su objetivo y justificar los resultados de su aplicación sobre la imagen de entrada facilitada (se debe tener en cuenta que la región de interés de la imagen que interesa realzar para verla con más detalle es la zona oscura del neumático que presenta muy bajo contraste).

Segunda Parte. Ecualización uniforme o igualación de histograma y modos de aplicación.

Ecualización global:

3. Implementar una función para ecualizar de forma uniforme una imagen. La función recibe como entrada una imagen de intensidad I y devuelve como salida la imagen Ieq ecualizada de forma uniforme. Se deben implementar tres versiones de la función (*ecualizar_imagen_v1*, *ecualizar_imagen_v2* y *ecualizar_imagen_v3*, todas definidas en

el fichero *Python funciones_ecualizar_imagen.py*) de acuerdo con los siguientes criterios:

- V1. Recorrido a nivel de píxel: implementación basada en el recorrido píxel a píxel de la imagen y calcular la función de transformación para cada píxel para generar la imagen de salida.
- V2. Tabla de transformación + indexación vectorizada: calcular la función de transformación para cada nivel de gris posible y aplicarla de forma global sobre la imagen para generar la imagen de salida.
- V3. Recorrido por nivel de gris: realizar el cálculo de la función de transformación para cada nivel de gris posible y, haciendo un barrido en los 256 posibles niveles de gris, aplicarla para generar la imagen de salida.

4. Aplicar las funciones anteriores para ecualizar la imagen de entrada, analizando el tiempo computacional de las distintas versiones, y comprobar que todas obtienen los mismos resultados. Visualizar el resultado y el histograma.

Ayuda de programación:

- Uso de la librería *time* (requiere *import time*) para medir tiempo computacional de ejecución de instrucciones Python:

```
start = time.time()
instrucciones…..
end = time.time()
print(f"Tiempo: {end - start:.6f} s")
```

Ecualización zonal:

5. Realizar una ecualización uniforme zonal sobre la imagen de entrada. Para ello, se ha de dividir la imagen en 9 sub-imágenes de las mismas dimensiones y aplicar la versión 1 de la función ecualizar a cada una de ellas. Finalmente, se debe componer la imagen final a partir de las sub-imágenes ecualizadas. Visualizar el resultado y el histograma.

Ayuda de programación:

- Usar las funciones de *numpy*, *hstack* y *vstack* para concatenar horizontalmente y verticalmente, respectivamente, matrices de la misma dimensión.

Ecualización local:

6. Implementar una función para ecualizar de forma local una imagen. La función recibe las siguientes variables de entrada:

   - Imagen de intensidad *I*: matriz *uint8* de entrada a ecualizar.
   - Matriz de vecindad *V*: matriz de "unos" que establece la vecindad con la que trabajará la función.
   - Opción de relleno *opcionRelleno*: variable tipo string para indicar el tipo de padding a aplicar, entre las opciones "*replicate*", "*zeros*" o "*reflect*".

   La función devuelve como salida la imagen ecualizada de forma local.

   La función debe aplicar una versión de la función *ecualizar_imagen*. Por ello, se deben definir en el fichero Python *funciones_ecualizar_imagen_local.py* tres versiones de la función, dependiendo de la versión de la función *ecualizar_imagen* con la que trabaje: *ecualizar_imagen_local_v1*, *ecualizar_imagen_local_v2* y *ecualizar_imagen_local_v3*.

7. Aplicar cada una de las versiones de la función *ecualizar_imagen_local* con la opción de padding "*replicate*" para ecualizar la imagen de entrada de forma local. Se debe considerar el siguiente tamaño de ventana de vecindad:

   - *Número de filas:* 1/3 del número de filas de la imagen; *Número de columnas:* 1/3 del número de columnas de la imagen. Se debe trabajar con un número de filas y columnas de la ventana de vecindad impar, por lo que, si no lo son, se debe forzar a que lo sean sumando o restando una unidad al resultado.

   Se debe analizar el tiempo computacional de las distintas versiones, y comprobar que todas obtienen los mismos resultados. Notar que el tiempo total de ejecución puede sobrepasar los 5 minutos, por lo que sólo debe hacerse una vez para analizar los tiempos. Visualizar el resultado y el histograma.

Ecualización local por ventanas:

8. A partir de la función *ecualizar_imagen_local_v2*, incluir en el fichero *Python funciones_ecualizar_imagen_local.py* una versión de la función llamada *ecualizar_imagen_local_por_ventanas.* Esta función no aplica una ecualización individual y específica para cada píxel. En esta nueva versión, la ecualización se aplicará por ventanas dadas por un nuevo parámetro de entrada *W*, tupla que indica el número de filas y columnas de estas ventanas.

9. Aplicar la función anterior sobre la imagen de entrada considerando que *W = (5,5)*. Mide el tiempo computacional y compáralo con el de la versión utilizada de la función *ecualizar_imagen_local_v2*.

10. Visualizar el resultado y el histograma junto con los de la función *ecualizar_imagen_local_v2*.

Observación:

- Se debe entender el objetivo y los fundamentos metodológicos de la técnica de ecualización de histograma, así como diferenciar y explicar los diferentes modos de aplicación expuestos en la práctica.

Tercera Parte. Caso de aplicación: ilustración de la técnica de especificación de histogramas para uniformizar el color de imágenes de fondo de ojo.

Cálculo de matriz lógica de píxeles de la retina:

11. Diseñar la función *calcular_ROI*, que reciba como entrada una imagen *RGB* de fondo de ojos y devuelva como salida una matriz lógica del mismo número de filas y columnas que localice los píxeles de interés de la imagen (píxeles de la retina). La función debe calcular esta máscara binaria de acuerdo con los siguientes criterios:
    - Un píxel es de retina si la suma de sus valores R, G y B es superior a 150.
    - Los falsos negativos (puntos a '0' binario dentro de la retina) deben eliminarse mediante la aplicación sucesiva de un filtro de máximos (implementarlo como una dilatación morfológica mediante la función *dilate* de *OpenCV*) y un filtro de mínimos (implementarlo como una erosión con la función *erode* de *OpenCV*) trabajando con la misma vecindad. Se debe trabajar con una vecindad completa *5x5*. Esta operación recibe el nombre de cierre morfológico (dilatación + erosión morfológica).
    - Los falsos positivos (puntos a '1' binario fuera de la retina) deben eliminarse mediante la aplicación sucesiva de un filtro de mínimos (*cv2.erode*) y un filtro de máximos (*cv2.dilate*) trabajando con la misma vecindad *5x5*, pero en este caso, tipo cruz. Esta operación recibe el nombre de apertura morfológica (erosión + dilatación morfológica).

12. Aplicar la función *calcular_ROI* a cada imagen de fondo de ojos facilitada en la práctica y visualiza en una misma ventana gráfica las imágenes y las máscaras binarias que calcula la función.

Aplicación de especificación de histogramas por canal:

13. Utilizando como histogramas de referencia los correspondientes a las componentes roja, verde y azul de la imagen *ColorPatron.bmp*, aplicar la función facilitada *especificar_imagen* (definida en fichero *Python funcion_especificar_imagen.py*) para uniformizar el color de las imágenes Color1.bmp, Color2.bmp, Color3.bmp y Color4.bmp. En la aplicación de la técnica, únicamente debe considerarse la información de los píxeles de la retina.

14. Para cada imagen analizada, representar en una ventana gráfica, dos filas de imágenes:
    - Fila superior: imagen patrón, imagen de entrada analizada e imagen salida del proceso de especificación de histograma.
    - Fila inferior: histogramas de los canales rojo, verde y azul de cada imagen de la fila superior, representados en la misma figura.

Observación:

- Se debe comprender el objetivo y los fundamentos metodológicos en los que se basa la técnica de especificación de histogramas; posteriormente, se debe analizar la función facilitada para la aplicación de la técnica a las imágenes de fondo de ojo con el objetivo de uniformizar su color.